

NaSalinha

Casos de Teste — Documentação Completa

Trainee: Carlos Vinicius Teixeira de Souza

Área 1 — Autenticação JWT

CT-AUTH-01	Login válido retorna tokens JWT (API — Positivo)
Request	login-usuario
Pré-cond.	Usuário carlosviniciusadmin@gmail.com cadastrado e ativo
Tipo	API
Passos	1. POST /api/auth/login com e-mail e senha corretos
Esperado	200 OK com accessToken e refreshToken
Obtido	200 OK com tokens retornados corretamente
Status	PASS

CT-AUTH-02	Login com senha incorreta deve retornar erro (API — Negativo)
Request	login-senha-invalida
Pré-cond.	Usuário pedro@example.com cadastrado
Tipo	API
Passos	1. POST /api/auth/login com senha errada "senha123Errada"
Esperado	401 Unauthorized
Obtido	401 Unauthorized
Status	PASS

CT-AUTH-03	Login com e-mail inexistente deve retornar erro (API — Negativo)
Request	login-sem-criar-conta
Pré-cond.	E-mail naoexiste@gmail.com não está cadastrado
Tipo	API
Passos	1. POST /api/auth/login com e-mail e senha de conta inexistente
Esperado	401 Unauthorized ou 404 Not Found
Obtido	401 Unauthorized
Status	PASS

CT-AUTH-04	Login com campos vazios deve retornar erro de validação (API — Negativo)
Request	login-campos-vazios
Pré-cond.	Nenhuma
Tipo	API
Passos	1. POST /api/auth/login com e-mail "" e password ""
Esperado	400 Bad Request com mensagem de validação
Obtido	400 Bad Request
Status	PASS

CT-AUTH-05	Login sem campo senha deve retornar erro de validação (API — Negativo)
Request	login-faltando-senha
Pré-cond.	Nenhuma
Tipo	API
Passos	1. POST /api/auth/login com apenas o campo e-mail no body
Esperado	400 Bad Request
Obtido	400 Bad Request
Status	PASS

CT-AUTH-06	Login com formato de e-mail inválido deve retornar erro (API — Negativo)
Request	login-formato-invalido-email
Pré-cond.	Nenhuma
Tipo	API
Passos	1. POST /api/auth/login com email "pedro" (sem @)
Esperado	400 Bad Request com mensagem de formato inválido
Obtido	400 Bad Request
Status	PASS

CT-AUTH-07	Tentativa de SQL Injection no login deve ser bloqueada (Segurança — Negativo)
Request	login-sql-injection
Pré-cond.	Nenhuma
Tipo	Segurança
Passos	1. POST /api/auth/login com email "" OR 1=1 --"
Esperado	400 ou 401 — payload malicioso não deve ser executado
Obtido	400/401 sem execução da injeção
Status	PASS

CT-AUTH-08	Cadastro válido deve criar usuário (API — Positivo)
Request	cadastrar-usuario
Pré-cond.	E-mail não cadastrado
Tipo	API
Passos	1. POST /api/auth/register com nome, e-mail, senha e role válidos
Esperado	201 Created — usuário criado com status PENDENTE, sem tokens retornados
Obtido	201 Created — API retorna accessToken e refreshToken imediatamente, sem verificação de e-mail
Status	FAIL
Bug Report	#1

CT-AUTH-09	Cadastro com e-mail já existente deve retornar erro (API — Negativo)
Request	cadastro-ja-cadastrado
Pré-cond.	E-mail bruno@gmail.com já cadastrado
Tipo	API
Passos	1. POST /api/auth/register com e-mail já existente
Esperado	409 Conflict ou 400 Bad Request
Obtido	409 Conflict
Status	PASS

CT-AUTH-10	Cadastro sem nome deve ser rejeitado (Funcional — Negativo)
Request	cadastro-sem-nome
Pré-cond.	Nenhuma
Tipo	Funcional
Passos	1. POST /api/auth/register com campo name vazio
Esperado	400 Bad Request
Obtido	400 Bad Request
Status	PASS

CT-AUTH-11	Cadastro com senha fraca deve ser rejeitado (Funcional — Negativo)	
Request	cadastro-senha-fraca	
Pré-cond.	Nenhuma	
Tipo	Funcional	
Passos	1. POST /api/auth/register com password "123"	
Esperado	400 Bad Request com mensagem sobre requisitos mínimos de senha	
Obtido	400 Bad Request	
Status	PASS	

CT-AUTH-12	Cadastro sem campo role deve atribuir role padrão ou retornar erro (API — Negativo)	
Request	cadastro-faltando-role-conta-nao-existe	
Pré-cond.	E-mail semrole@gmail.com não cadastrado	
Tipo	API	
Passos	1. POST /api/auth/register sem o campo role	
Esperado	201 com role padrão atribuída, ou 400 informando campo obrigatório	
Obtido	Conta criada com padrão MEMBER	
Status	PASS	

CT-AUTH-13	Cadastro com role inexistente deve ser rejeitado (API — Negativo)	
Request	cadastro-com-role-inexistente	
Pré-cond.	Nenhuma	
Tipo	API	
Passos	1. POST /api/auth/register com role "PRESIDENTE"	
Esperado	400 Bad Request — role inválida não deve ser aceita	
Obtido	400 Bad Request	
Status	PASS	

CT-AUTH-14	Cadastro com strings espaçadas deve ser sanitizado (Funcional — Negativo)	
Request	cadastro-com-string-espacada / cadastro-com-email-e-role-espacado	
Pré-cond.	Nenhuma	
Tipo	Funcional	
Passos	1. POST /api/auth/register com espaços extras em nome, e-mail e role	
Esperado	Sistema deve sanitizar (trim) os campos ou retornar erro de validação	
Obtido	String é sanitizada	
Status	PASS	

CT-AUTH-15	Recuperação de senha com e-mail inexistente não deve revelar conta (Segurança — Negativo)
Request	email_nao_existe
Pré-cond.	E-mail carlosviniciusNAOEXISTE@gmail.com não cadastrado
Tipo	Segurança
Passos	1. POST /api/auth/forgot-password com e-mail inexistente
Esperado	200 OK com mensagem genérica (não deve confirmar nem negar existência do e-mail)
Obtido	200 OK com mensagem genérica
Status	PASS

CT-AUTH-16	Usuário não-admin não deve acessar endpoint restrito de seasons (API — Negativo)
Request	logando_sem_ser_admin / get_seasons_sendo_member
Pré-cond.	Usuários com roles MEMBER e TRAINEE autenticados
Tipo	API
Passos	1. GET /api/seasons com token de usuário não-admin
Esperado	403 Forbidden
Obtido	200 OK com dados retornados
Status	FAIL
Bug Report	#2

CT-AUTH-17	Usuário MEMBER não deve acessar endpoint de listagem de usuários (API — Negativo)
Request	acessar_api_sendo_member
Pré-cond.	Usuário autenticado com role MEMBER
Tipo	API
Passos	1. GET /api/users com token de MEMBER
Esperado	403 Forbidden
Obtido	"Sem permissao para acessar esse recurso"
Status	PASS

CT-AUTH-18	Senha não deve ser retornada em texto plano na listagem de usuários (Segurança)
Request	check_senha_hash
Pré-cond.	Usuário ADMIN autenticado
Tipo	Segurança
Passos	1. GET /api/users com token de ADMIN; 2. Verificar campo password na resposta
Esperado	Campo password deve estar hashado (bcrypt) ou ausente na resposta
Obtido	Campo password não aparece na resposta da API
Status	Indefinido

CT-AUTH-19	Regressão: após correção dos bugs de autenticação e autorização (Regressão)
Pré-cond.	Correções dos bugs #1 e #2 implementadas
Tipo	Regressão
Cenário	Cadastro exige verificação de e-mail; endpoints restritos bloqueiam roles não autorizadas
Testes	1. Cadastrar novo usuário — tokens não devem ser retornados antes da verificação; 2. Login com e-mail não verificado — deve ser bloqueado; 3. Login válido após verificação — deve retornar tokens; 4. GET /api/seasons com token MEMBER ou TRAINEE — deve retornar 403; 5. GET /api/seasons com token ADMIN — deve retornar 200 OK
Objetivo	Garantir que a correção do fluxo de cadastro e do RBAC não afetou o login válido nem o acesso de administradores
Status	Aguardando correção

Área 2 — Check-in por Foto

CT-CHECKIN-01	Check-in com imagem válida deve ser criado com status PENDENTE (Funcional — Positivo)
Request	enviar_foto
Pré-cond.	Usuário autenticado; nenhum check-in realizado nas últimas 24h
Tipo	Funcional
Passos	1. POST /api/checkins com imagem .png válida e token JWT
Esperado	201 Created — check-in criado com status PENDENTE, sem pontos atribuídos
Obtido	201 Created — check-in criado com status APPROVED e pontos atribuídos imediatamente
Status	FAIL
Bug Report	#4

CT-CHECKIN-02	Check-in sem foto deve ser rejeitado (API — Negativo)	
Request	enviar_sem_foto	
Pré-cond.	Usuário autenticado	
Tipo	API	
Passos	1. POST /api/checkins sem arquivo no campo foto	
Esperado	400 Bad Request — campo foto obrigatório	
Obtido	400 Bad Request — campo foto obrigatório	
Status	PASS	

CT-CHECKIN-03	Check-in com formato de arquivo inválido deve ser rejeitado (API — Negativo)	
Request	enviar_formato_incorreto	
Pré-cond.	Usuário autenticado	
Tipo	API	
Passos	1. POST /api/checkins com arquivo .cpp no campo foto	
Esperado	400 Bad Request — apenas imagens devem ser aceitas	
Obtido	400 Bad Request — apenas imagens devem ser aceitas	
Status	PASS	

CT-CHECKIN-04	Consulta de check-ins deve retornar apenas os do usuário autenticado (API — Positivo)	
Request	checar_checkin	
Pré-cond.	Usuário autenticado com ao menos um check-in realizado	
Tipo	API	
Passos	1. GET /api/checkins/ com token JWT	
Esperado	200 OK com lista de check-ins pertencentes ao usuário autenticado	
Obtido	Retorna todos os check-ins (de todos os usuários)	
Status	FAIL	

CT-CHECKIN-05	Usuário não deve acessar check-in de outro usuário (API — Negativo)
Request	checar_checkin_outra_pessoa_sem_admin
Pré-cond.	Usuário autenticado; ID de check-in pertencente a outro usuário conhecido
Tipo	API
Passos	1. GET /api/checkins/{id} com ID de check-in alheio e token de usuário não-admin
Esperado	403 Forbidden ou 404 Not Found
Obtido	200 OK com dados completos do check-in alheio
Status	FAIL
Bug Report	#6

CT-CHECKIN-06	Remoção de check-in próprio deve funcionar (API — Positivo)
Request	deletar_foto
Pré-cond.	Usuário autenticado com check-in próprio existente
Tipo	API
Passos	1. DELETE /api/checkins/{id} com ID de check-in próprio
Esperado	200 ou 204 — check-in removido com sucesso; pontos subtraídos
Obtido	Check-in removido com sucesso; pontos e contagem permanecem inalterados
Status	FAIL
Bug Report	#8

CT-CHECKIN-07	Admin deve poder remover check-in de qualquer usuário (API — Positivo)
Request	deletar_foto_admin
Pré-cond.	Usuário ADMIN autenticado; check-in de outro usuário existente
Tipo	API
Passos	1. DELETE /api/checkins/{id} com token de ADMIN e ID de check-in de outro usuário
Esperado	200 ou 204 — check-in removido com sucesso
Obtido	Check-in é deletado, mas pontos não são recomputados (permanece inalterado)
Status	FAIL
Bug Report	#8

CT-CHECKIN-08	Endpoint de alteração de status de check-in deve existir (Funcional — Positivo)
Request	PATCH /checkins/:id (New Request)
Pré-cond.	Check-in existente; usuário ADMIN autenticado
Tipo	Funcional
Passos	1. PATCH /checkins/:id com body de atualização de status
Esperado	200 OK — status atualizado para APPROVED ou REJECTED
Obtido	Rota não encontrada — endpoint não implementado
Status	FAIL
Bug Report	#5

CT-CHECKIN-09	Sistema não deve permitir múltiplos check-ins no mesmo dia (API — Negativo)
Request	enviar_foto (executado múltiplas vezes)
Pré-cond.	Usuário autenticado com check-in já realizado no dia
Tipo	API
Passos	1. POST /api/checkins repetido no mesmo dia
Esperado	400 ou 409 a partir do segundo check-in
Obtido	201 Created — múltiplos check-ins aceitos no mesmo dia
Status	FAIL
Bug Report	#3

CT-CHECKIN-10	Regressão: após correção dos bugs de check-in, fluxo completo deve respeitar regras de negócio (Regressão)
Pré-cond.	Correções dos bugs #3, #4, #5 e #6 implementadas
Tipo	Regressão
Cenário	Check-in criado com status PENDENTE; endpoint de moderação disponível; restrição de 24h ativa; acesso IDOR bloqueado
Testes	1. POST /api/checkins com imagem válida — status deve ser PENDENTE; 2. POST /api/checkins no mesmo dia — deve ser bloqueado com 400 ou 409; 3. PATCH /checkins/:id com ADMIN — deve atualizar status para APPROVED ou REJECTED; 4. GET /api/checkins/{id} com token de outro usuário sem ser ADMIN — deve retornar 403 ou 404; 5. DELETE /api/checkins/{id} com token de dono do check-in — deve remover com sucesso
Objetivo	Garantir que as correções do módulo de check-in não causaram regressão nos fluxos de envio válido, listagem e remoção
Status	Aguardando correção

CT-CHECKIN-X X	Ad-hoc — Código de upload em disco não é utilizado em produção (Funcional / Análise)	
Pré-cond.	Acesso ao código-fonte do projeto; ambiente rodando via Docker	
Tipo	Funcional / Análise de Código	
Passos	1. Inspecionar o código de configuração de upload; 2. Verificar se há distinção de ambientes (dev/prod) no Dockerfile e docker-compose.yml; 3. Verificar se o código de upload em disco é referenciado em algum fluxo ativo	
Esperado	Dockerfile e docker-compose devem definir builds distintas para desenvolvimento e produção; código de upload em disco deve estar ativo apenas no ambiente correto	
Obtido	Nenhuma distinção de ambiente encontrada nos arquivos Docker; código de upload em disco presente mas inutilizado	
Status	FAIL	
Bug Report	#7	

Área 3 — Sistema de Pontos

CT-PONTOS-0 1	Ranking deve ser acessível a usuários autenticados (API — Positivo)	
Request	checar_pontos	
Pré-cond.	Usuário autenticado com token JWT válido	
Tipo	API	
Passos	1. GET /api/rankings com token JWT	
Esperado	200 OK com lista de usuários ordenada por pontuação (crescente)	
Obtido	200 OK com lista de usuários ordenada por pontuação (crescente)	
Status	PASS	

CT-PONTOS-0 2	Pontos devem ser atribuídos apenas após aprovação do check-in (Funcional — Negativo)	
Request	enviar_foto + checar_pontos	
Pré-cond.	Usuário autenticado sem check-ins anteriores	
Tipo	Funcional	
Passos	1. POST /api/checkins com imagem; 2. GET /api/rankings imediatamente após	
Esperado	Pontos não devem ser atribuídos antes da aprovação por ADMINISTRADOR	
Obtido	Pontos atribuídos imediatamente na criação do check-in	
Status	FAIL	
Bug Report	#4	

CT-PONTOS-03	Remoção de check-in deve subtrair os pontos correspondentes (API — Negativo)
Request	deletar_foto + checar_pontos
Pré-cond.	Usuário com check-in criado e pontos acumulados
Tipo	API
Passos	1. Verificar pontos antes; 2. DELETE /api/checkins/{id}; 3. Verificar pontos após
Esperado	Saldo reduzido; contagem decrementada; ranking atualizado
Obtido	Check-in removido, pontos e contagem permanecem inalterados
Status	FAIL
Bug Report	#8

CT-PONTOS-04	Múltiplos check-ins no mesmo dia não devem acumular pontos (API — Negativo)
Request	enviar_foto (múltiplas execuções) + checar_pontos
Pré-cond.	Usuário autenticado
Tipo	API
Passos	1. Realizar 3 check-ins consecutivos; 2. GET /api/rankings
Esperado	Apenas o primeiro check-in do dia gera pontos
Obtido	Cada check-in adicional incrementa os pontos, corrompendo o ranking
Status	FAIL
Bug Report	#3

CT-PONTOS-05	Regressão: após correção da consistência de pontos, saldo deve ser preciso em todo o ciclo (Regressão)
Pré-cond.	Correções dos bugs #4, #8 e #3 implementadas
Tipo	Regressão
Cenário	Pontos atribuídos apenas após aprovação; subtraídos ao deletar check-in
Testes	1. Criar check-in — pontos não devem ser atribuídos, deve manter status PENDENTE; 2. ADMIN aprovar/rejeitar check-in — pontos devem ser atribuídos ou mantidos; 3. Deletar check-in aprovado — pontos devem ser subtraídos; 4. Consultar GET /api/rankings — posição do usuário deve estar atualizada; 5. Segundo check-in no mesmo dia — bloqueado, pontos inalterados
Objetivo	Garantir que a correção da atribuição de pontos não afetou os fluxos de criação, aprovação, remoção e ranking
Status	Aguardando correção

Área 4 — Extras (Ad-hoc / Frontend)

CT-FRONTEND-01	Ranking de pontuação deve ser visível na interface para todos os usuários autenticados (Ad-hoc)
Tipo	Funcional / Interface
Pré-cond.	Usuário autenticado com qualquer role
Passos	1. Fazer login com qualquer usuário; 2. Navegar pela interface em busca da tela de ranking
Esperado	Tela de ranking deve estar acessível na interface para qualquer usuário autenticado
Obtido	Nenhuma tela de ranking existe no frontend; a pontuação só é consultável via requisição direta à API
Status	FAIL
Bug Report	#10

CT-FRONTEND-02	Sistema de pontos deve refletir decrementos na interface (Ad-hoc)
Tipo	Funcional / Interface
Pré-cond.	Usuário autenticado com pontos acumulados; ao menos um check-in aprovado
Passos	1. Verificar saldo de pontos na interface; 2. Deletar um check-in aprovado; 3. Verificar saldo após a remoção
Esperado	Saldo de pontos deve ser decrementado e o ranking atualizado na tela
Obtido	Saldo de pontos permanece inalterado na interface após remoção do check-in (condiz com CT-PONTOS-03)
Status	FAIL
Bug Report	#11

Área 5 — Temporadas (CRUD + Validações)

CT-TEMPORAS-01	Criação de Temporada (ADMIN vs MEMBER)
Request	POST /api/seasons
Pré-cond.	Usuário autenticado (ADMIN e MEMBER)
Tipo	API Authorization
Passos	1. Criar temporada como ADMIN; 2. Tentar como MEMBER
Esperado	ADMIN cria; MEMBER não possui permissão
Obtido	Comportamento conforme esperado
Status	PASS

CT-TEMPORA DAS-02	Leitura de Temporada por ID
Request	GET /api/seasons/{id}
Pré-cond.	Temporada existente
Tipo	API
Passos	1. Criar temporada; 2. Buscar pelo ID
Esperado	Dados retornados corretamente
Obtido	Dados corretos retornados
Status	PASS

CT-TEMPORA DAS-03	Atualização de Temporada (PUT)
Request	PUT /api/seasons/{id}
Pré-cond.	Usuário ADMIN + temporada existente
Tipo	API
Passos	1. Criar temporada; 2. Atualizar via PUT
Esperado	Dados atualizados corretamente
Obtido	Atualização realizada com sucesso
Status	PASS

CT-TEMPORA DAS-04	Atualização Parcial (PATCH)
Request	PATCH /api/seasons/{id}
Pré-cond.	Usuário ADMIN
Tipo	API
Passos	1. Alterar campo isActive
Esperado	Campo atualizado corretamente
Obtido	Atualização realizada com sucesso
Status	PASS

CT-TEMPORA DAS-05	Deleção de Temporada (ADMIN vs MEMBER)
Request	DELETE /api/seasons/{id}
Pré-cond.	Temporada existente
Tipo	API Authorization
Passos	1. Tentar deletar como MEMBER; 2. Deletar como ADMIN
Esperado	Apenas ADMIN pode deletar
Obtido	Comportamento correto
Status	PASS

CT-TEMPORA DAS-06	Check-in sem temporada ativa
Request	POST /api/checkins
Pré-cond.	Nenhuma temporada ativa
Tipo	API Validation
Passos	1. Garantir todas inativas; 2. Enviar check-in
Esperado	400 com mensagem 'nenhuma temporada ativa'
Obtido	400 retornado corretamente
Status	PASS

CT-TEMPORA DAS-07	Datas inválidas geram epoch (1970)
Request	POST /api/seasons
Pré-cond.	Usuário ADMIN
Tipo	API Validation
Passos	1. Enviar datas inválidas
Esperado	400 Bad Request
Obtido	Datas convertidas para 1970
Status	FAIL
Bug Report	#7

CT-TEMPORA DAS-08	Frontend exibe múltiplas temporadas ativas
Request	GET /api/seasons
Pré-cond.	Múltiplas seasons com isActive=true
Tipo	Frontend
Passos	1. Criar múltiplas seasons ativas; 2. Abrir UI
Esperado	Apenas uma ativa visível
Obtido	Frontend mostra várias como ativas
Status	FAIL
Bug Report	#5